SQL INTERVIEW SYNTAX SHEET — FROM YOUR NOTES

---

1. DATABASE COMMANDS

Create database

```
CREATE DATABASE database_name;
```

Show databases

```
SHOW DATABASES;
```

Select database

```
USE database_name;
```

Delete database

```
DROP DATABASE database_name;
```

---

2. TABLE COMMANDS — DDL

Create table

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype
);
```

Example

```
CREATE TABLE employees (
    emp_id INT,
    emp_name VARCHAR(50),
    salary DECIMAL(10,2),
    department VARCHAR(30)
);
```

Show tables

```
SHOW TABLES;
```

Describe table

```
DESC table_name;
```

or

```
DESCRIBE table_name;
```

---

3. ALTER TABLE

Add column

```
ALTER TABLE table_name
ADD COLUMN column_name datatype;
```

Add column after another column

```
ALTER TABLE table_name
ADD COLUMN column_name datatype
AFTER existing_column;
```

Modify column datatype

```
ALTER TABLE table_name
MODIFY COLUMN column_name new_datatype;
```

Rename column

```
ALTER TABLE table_name
RENAME COLUMN old_column_name TO new_column_name;
```

Change column name + datatype

```
ALTER TABLE table_name
CHANGE COLUMN old_column_name new_column_name datatype;
```

Drop column

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

Rename table

```
RENAME TABLE old_table_name TO new_table_name;
```

---

4. DROP, DELETE, TRUNCATE

Drop table

```
DROP TABLE table_name;
```

Deletes the table structure + data.

Delete selected records

```
DELETE FROM table_name
WHERE condition;
```

Delete all records

```
DELETE FROM table_name;
```

Truncate table

```
TRUNCATE TABLE table_name;
```

Removes all rows but keeps the table structure.

Interview difference

Command	Data	Structure
DELETE	Deletes selected/all rows	Keeps
TRUNCATE	Deletes all rows	Keeps
DROP	Deletes everything	Deletes

---

5. INSERT — DML

Insert specific columns

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

Insert all columns

```
INSERT INTO table_name
VALUES (value1, value2, value3);
```

Insert multiple rows

```
INSERT INTO table_name (column1, column2)
VALUES
(value1, value2),
(value3, value4),
(value5, value6);
```

6. SELECT — DQL

Select all records

```
SELECT *
FROM table_name;
```

Select specific columns

```
SELECT column1, column2
FROM table_name;
```

Alias

```
SELECT column_name AS alias_name
FROM table_name;
```

---

7. DISTINCT

Used to remove duplicate values.

```
SELECT DISTINCT column_name
```

```
FROM table_name;
```

Multiple columns

```
SELECT DISTINCT column1, column2
FROM table_name;
```

---

8. WHERE CLAUSE

```
SELECT *
FROM table_name
WHERE condition;
```

Example:

```
SELECT *
FROM employees
WHERE salary > 50000;
```

---

9. COMPARISON OPERATORS

```
=       Equal
<>      Not equal
!=      Not equal
>       Greater than
<       Less than
>=      Greater than or equal
<=      Less than or equal
```

Syntax

```
SELECT *
FROM employees
WHERE salary > 50000;
SELECT *
FROM employees
WHERE salary = 50000;
SELECT *
FROM employees
WHERE salary <> 50000;
```

---

10. LOGICAL OPERATORS

AND

Both conditions must be true.

```
SELECT *
FROM employees
WHERE department = 'IT'
AND salary > 50000;
```

OR

At least one condition must be true.

```
SELECT *
FROM employees
WHERE department = 'IT'
OR department = 'HR';
```

NOT

```
SELECT *
FROM employees
WHERE NOT department = 'HR';
```

---

11. BETWEEN

Used to check a range.

```
SELECT *
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Example:

```
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 70000;
```

---

12. IN

Used to match multiple values.

```
SELECT *
FROM table_name
WHERE column_name IN (value1, value2, value3);
```

Example:

```
SELECT *
FROM employees
WHERE department IN ('IT', 'HR', 'Finance');
```

NOT IN

```
SELECT *
FROM employees
WHERE department NOT IN ('IT', 'HR');
```

---

13. LIKE

Very important in interviews.

Starts with

```
SELECT *
FROM employees
WHERE emp_name LIKE 'A%';
```

Ends with

```
SELECT *
FROM employees
WHERE emp_name LIKE '%A';
```

Contains

```
SELECT *
FROM employees
WHERE emp_name LIKE '%A%';
```

Second character is A

```
SELECT *
FROM employees
WHERE emp_name LIKE '_A%';
```

Wildcards

```
%  → Zero or more characters
_  → Exactly one character
```

---

14. NULL OPERATORS

Find NULL values

```
SELECT *
FROM table_name
WHERE column_name IS NULL;
```

Find NOT NULL values

```
SELECT *
FROM table_name
WHERE column_name IS NOT NULL;
```

⚠️ Don't write:

```
WHERE column_name = NULL;
```

Use `IS NULL`.

---

15. ORDER BY

Ascending

```
SELECT *
FROM table_name
ORDER BY column_name ASC;
```

Descending

```
SELECT *
FROM table_name
ORDER BY column_name DESC;
```

Multiple columns

```
SELECT *
FROM employees
ORDER BY department ASC, salary DESC;
```

---

16. LIMIT

```
SELECT *
FROM table_name
LIMIT number;
```

Example:

```
SELECT *
FROM employees
LIMIT 5;
```

Pagination

```
SELECT *
FROM employees
LIMIT 5 OFFSET 10;
```

---

17. MATHEMATICAL / ARITHMETIC QUERIES

You can directly perform calculations inside `SELECT`.

```
SELECT column1 + column2
FROM table_name;
```

Addition

```
SELECT salary + 5000 AS new_salary
FROM employees;
```

Subtraction

```
SELECT salary - 5000 AS reduced_salary
FROM employees;
```

Multiplication

```
SELECT price * quantity AS total
FROM products;
```

Division

```
SELECT salary / 12 AS monthly_salary
FROM employees;
```

Percentage

```
SELECT salary + (salary * 0.10) AS increased_salary
FROM employees;
```

Tax

```
SELECT price,
       price + (price * 0.12) AS price_with_tax
FROM products;
```

---

18. UPDATE

Update one column

```
UPDATE table_name
SET column_name = new_value
WHERE condition;
```

Example:

```
UPDATE employees
SET salary = 60000
WHERE emp_id = 101;
```

Update multiple columns

```
UPDATE employees
SET salary = 60000,
    department = 'IT'
WHERE emp_id = 101;
```

Update using calculation

```
UPDATE employees
SET salary = salary + 5000
WHERE department = 'IT';
```

⚠️ **Interview tip:** Always check the `WHERE` condition before executing `UPDATE`.

---

19. DELETE

Delete specific record

```
DELETE FROM table_name
WHERE condition;
```

Example:

```
DELETE FROM employees
WHERE emp_id = 101;
```

Delete based on condition

```
DELETE FROM employees
WHERE salary < 30000;
```

---

20. AGGREGATE FUNCTIONS

Your notes cover:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

COUNT

```
SELECT COUNT(*)
FROM employees;
```

Count a column

```
SELECT COUNT(column_name)
FROM table_name;
```

Remember:

- `COUNT(*)` counts rows.
- `COUNT(column)` ignores `NULL`.

SUM

```
SELECT SUM(salary)
```

```
FROM employees;
```

AVG

```
SELECT AVG(salary)
FROM employees;
```

MIN

```
SELECT MIN(salary)
FROM employees;
```

MAX

```
SELECT MAX(salary)
FROM employees;
```

---

21. GROUP BY

Very important.

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

Example:

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

Count employees department-wise

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department;
```

Maximum salary department-wise

```
SELECT department, MAX(salary)
FROM employees
GROUP BY department;
```

---

22. HAVING

`WHERE` filters rows.

`HAVING` filters groups.

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 50000;
```

Interview pattern

```
SELECT column, aggregate_function(column)
FROM table
WHERE condition
GROUP BY column
HAVING aggregate_function(column) condition;
```

---

23. SQL QUERY EXECUTION ORDER

Very important for interviews:

```
FROM
JOIN
WHERE
GROUP BY
HAVING
SELECT
DISTINCT
ORDER BY
LIMIT
```

You **write** SQL differently, but logically SQL processes these clauses in this order.

---

24. NUMERIC FUNCTIONS

From your notes:

ABS

```
SELECT ABS(-100);
```

CEIL

```
SELECT CEIL(12.34);
```

FLOOR

```
SELECT FLOOR(12.34);
```

ROUND

```
SELECT ROUND(12.3456);
```

Round to 2 decimal places

```
SELECT ROUND(12.3456, 2);
```

MOD

```
SELECT MOD(17, 5);
```

POWER

```
SELECT POW(2, 3);
```

SQRT

```
SELECT SQRT(25);
```

---

25. DATE & TIME FUNCTIONS

Current date

```
SELECT CURDATE();
```

Current time

```
SELECT CURTIME();
```

Current date + time

```
SELECT NOW();
```

Extract date

```
SELECT DATE(datetime_column)
```

```
FROM table_name;
```

Extract time

```
SELECT TIME(datetime_column)
FROM table_name;
```

Year

```
SELECT YEAR(date_column)
FROM table_name;
```

Month

```
SELECT MONTH(date_column)
FROM table_name;
```

Day

```
SELECT DAY(date_column)
FROM table_name;
```

Week

```
SELECT WEEK(date_column)
FROM table_name;
```

Day of week

```
SELECT DAYOFWEEK(date_column)
FROM table_name;
```

Add date

```
SELECT DATE_ADD(date_column, INTERVAL 10 DAY)
FROM table_name;
```

Subtract date

```
SELECT DATE_SUB(date_column, INTERVAL 10 DAY)
FROM table_name;
```

---

26. JOINS

INNER JOIN

Returns matching records from both tables.

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.common_column = table2.common_column;
```

---

27. LEFT JOIN

Returns all rows from the left table and matching rows from right table.

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.common_column = table2.common_column;
```

---

28. RIGHT JOIN

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.common_column = table2.common_column;
```

---

29. CROSS JOIN

Every row of first table combines with every row of second table.

```
SELECT *
FROM table1
CROSS JOIN table2;
```

---

30. SELF JOIN

A table joined with itself.

```
SELECT e1.column, e2.column
FROM employees e1
JOIN employees e2
ON e1.common_column = e2.common_column;
```

Classic manager-employee example

```
SELECT e.emp_name AS employee,
       m.emp_name AS manager
FROM employees e
JOIN employees m
ON e.manager_id = m.emp_id;
```

---

31. FULL OUTER JOIN

MySQL doesn't directly support `FULL OUTER JOIN`.

Common approach:

```
SELECT *
FROM table1
LEFT JOIN table2
ON table1.id = table2.id

UNION

SELECT *
FROM table1
RIGHT JOIN table2
ON table1.id = table2.id;
```

---

32. SUBQUERIES

A query inside another query.

Basic subquery

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

---

33. SINGLE-ROW SUBQUERY

Returns one value.

```
SELECT *
```

```
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
);
```

---

34. MULTI-ROW SUBQUERY — IN

```
SELECT *
FROM employees
WHERE department_id IN (
    SELECT department_id
    FROM departments
    WHERE location = 'Hyderabad'
);
```

---

35. ANY / SOME

Condition must be true for **at least one** value.

```
SELECT *
FROM employees
WHERE salary > ANY (
    SELECT salary
    FROM employees
    WHERE department = 'HR'
);
```

`ANY` and `SOME` are interchangeable.

---

36. ALL

Condition must be true for **all** returned values.

```
SELECT *
FROM employees
WHERE salary > ALL (
    SELECT salary
    FROM employees
    WHERE department = 'HR'
);
```

Remember:

```
> ALL → greater than maximum
< ALL → less than minimum
```

---

37. EXISTS

Checks whether the subquery returns any row.

```
SELECT *
FROM departments d
WHERE EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
);
```

NOT EXISTS

```
SELECT *
FROM departments d
WHERE NOT EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
);
```

---

38. CORRELATED SUBQUERY

Inner query refers to the outer query.

```
SELECT e1.*
FROM employees e1
WHERE salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.department = e1.department
);
```

This is a **very good interview example**.

---

39. SECOND HIGHEST SALARY

One of the most commonly asked SQL interview questions.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

---

40. SECOND LOWEST SALARY

```
SELECT MIN(salary) AS second_lowest
FROM employees
WHERE salary > (
    SELECT MIN(salary)
    FROM employees
);
```

---

41. THIRD HIGHEST SALARY

Using nested `MAX()`:

```
SELECT MAX(salary) AS third_highest
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
    WHERE salary < (
        SELECT MAX(salary)
        FROM employees
    )
);
```

---

42. NTH HIGHEST SALARY — INTERVIEW FAVORITE

Using `DISTINCT` + `LIMIT`:

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

Here:

```
1st highest → OFFSET 0
```

```
2nd highest → OFFSET 1
3rd highest → OFFSET 2
Nth highest → OFFSET N-1
```

---

43. SECOND HIGHEST SALARY DEPARTMENT-WISE

Pattern from your notes:

```
SELECT department,
       MAX(salary) AS second_highest
FROM employees e
WHERE salary < (
    SELECT MAX(salary)
    FROM employees e2
    WHERE e2.department = e.department
)
GROUP BY department;
```

---

44. CONSTRAINTS

NOT NULL

```
CREATE TABLE employees (
    emp_id INT,
    emp_name VARCHAR(50) NOT NULL
);
```

UNIQUE

```
CREATE TABLE employees (
    email VARCHAR(100) UNIQUE
);
```

PRIMARY KEY

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY
);
```

AUTO_INCREMENT

```
CREATE TABLE employees (
```

```
    emp_id INT AUTO_INCREMENT PRIMARY KEY
);
```

DEFAULT

```
CREATE TABLE employees (
    status VARCHAR(20) DEFAULT 'ACTIVE'
);
```

CHECK

```
CREATE TABLE employees (
    age INT CHECK (age >= 18)
);
```

FOREIGN KEY

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    department_id INT,
    FOREIGN KEY (department_id)
        REFERENCES departments(department_id)
);
```

---

45. COMPOSITE PRIMARY KEY

```
CREATE TABLE student_course (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id)
);
```

---

46. TCL — TRANSACTION CONTROL

Start transaction

```
START TRANSACTION;
```

or

```
BEGIN;
```

Commit

```
COMMIT;
```

Rollback

```
ROLLBACK;
```

Savepoint

```
SAVEPOINT savepoint_name;
```

Rollback to savepoint

```
ROLLBACK TO SAVEPOINT savepoint_name;
```

Release savepoint

```
RELEASE SAVEPOINT savepoint_name;
```

Interview example

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 5000
WHERE account_id = 101;

UPDATE accounts
SET balance = balance + 5000
WHERE account_id = 102;

COMMIT;
```

If something fails:

```
ROLLBACK;
```

---

47. DCL

GRANT

```
GRANT privilege
ON database_name.table_name
TO 'username'@'localhost';
```

Example:

```
GRANT SELECT, INSERT
ON pfs4.employees
TO 'user1'@'localhost';
```

REVOKE

```
REVOKE privilege
ON database_name.table_name
FROM 'username'@'localhost';
```

---

48. CREATE USER

```
CREATE USER 'username'@'localhost'
IDENTIFIED BY 'password';
```

Drop user

```
DROP USER 'username'@'localhost';
```

---

49. VIEWS

Your notes also cover views.

Create view

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example:

```
CREATE VIEW high_salary_employees AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > 50000;
```

View data

```
SELECT *
FROM high_salary_employees;
```

Alter view

```
ALTER VIEW view_name AS
SELECT ...
FROM ...;
```

Drop view

```
DROP VIEW view_name;
```

---

50. STORED PROCEDURES

Procedure without parameters

```
DELIMITER //

CREATE PROCEDURE procedure_name()
BEGIN
    SELECT *
    FROM employees;
END //

DELIMITER ;
```

Call:

```
CALL procedure_name();
```

---

51. STORED PROCEDURE — IN PARAMETER

```
DELIMITER //

CREATE PROCEDURE getEmployee(IN eid INT)
BEGIN
    SELECT *
    FROM employees
    WHERE emp_id = eid;
END //

DELIMITER ;
```

Call:

```
CALL getEmployee(101);
```

---

52. PROCEDURE WITH MULTIPLE PARAMETERS

```
DELIMITER //

CREATE PROCEDURE salaryRange(
    IN minSalary DECIMAL(10,2),
    IN maxSalary DECIMAL(10,2)
)
BEGIN
    SELECT *
    FROM employees
    WHERE salary BETWEEN minSalary AND maxSalary;
END //

DELIMITER ;
```

Call:

```
CALL salaryRange(40000, 70000);
```

---

53. OUT PARAMETER

```
DELIMITER //

CREATE PROCEDURE totalEmployees(OUT total INT)
BEGIN
    SELECT COUNT(*)
    INTO total
    FROM employees;
END //

DELIMITER ;
```

Call:

```
CALL totalEmployees(@total);

SELECT @total;
```

---

54. INOUT PARAMETER

General syntax:

```
DELIMITER //
```

```
CREATE PROCEDURE procedure_name(
    INOUT parameter_name datatype
)
BEGIN
    -- logic
END //

DELIMITER ;
```

Call:

```
SET @value = 10;

CALL procedure_name(@value);

SELECT @value;
```

---

55. SHOW PROCEDURES

```
SHOW PROCEDURE STATUS;
```

---

56. DROP PROCEDURE

```
DROP PROCEDURE procedure_name;
```

---

57. TRIGGERS

Very important if asked about database automation.

General syntax

```
DELIMITER //

CREATE TRIGGER trigger_name
BEFORE INSERT
ON table_name
FOR EACH ROW
BEGIN
    -- trigger logic
END //

DELIMITER ;
```

The event can be:

```
INSERT
UPDATE
DELETE
```

Timing:

```
BEFORE
AFTER
```

---

58. NEW AND OLD

NEW

Used with:

```
INSERT
UPDATE
```

Refers to the new value.

OLD

Used with:

```
UPDATE
DELETE
```

Refers to the previous value.

---

59. TRIGGER EXAMPLE

Prevent salary below 20,000:

```
DELIMITER //

CREATE TRIGGER check_salary
BEFORE INSERT
ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 20000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be less than 20000';
```

```
    END IF;
END //

DELIMITER ;
```

---

60. IMPORTANT INTERVIEW QUERY PATTERNS

These are the ones I strongly recommend memorizing.

Employees earning more than average salary

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Highest salary

```
SELECT MAX(salary)
FROM employees;
```

Lowest salary

```
SELECT MIN(salary)
FROM employees;
```

Second highest

```
SELECT MAX(salary)
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Employees with same department as Ravi

```
SELECT *
FROM employees
WHERE department = (
    SELECT department
    FROM employees
    WHERE emp_name = 'Ravi'
);
```

Duplicate values

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name
HAVING COUNT(*) > 1;
```

Departments having more than 5 employees

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

Top 5 salaries

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

Employees whose name starts with A

```
SELECT *
FROM employees
WHERE emp_name LIKE 'A%';
```

Employees who don't belong to HR

```
SELECT *
FROM employees
WHERE department <> 'HR';
```

Employees with salary between 40K and 70K

```
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 70000;
```

⭐ MOST IMPORTANT FOR YOUR INTERVIEW

If you have limited time, revise in this order:

```
1. SELECT + WHERE
2. INSERT
```

```
3. UPDATE
4. DELETE
5. ORDER BY
6. GROUP BY
7. HAVING
8. Aggregate Functions
9. LIKE
10. IN / BETWEEN
11. NULL
12. JOINS
13. Subqueries
14. Second Highest Salary
15. Constraints
16. DELETE vs TRUNCATE vs DROP
17. TCL
18. Views
19. Stored Procedures
20. Triggers
```

One-line memory trick

```
DDL → CREATE, ALTER, DROP, TRUNCATE
DML → INSERT, UPDATE, DELETE
DQL → SELECT
DCL → GRANT, REVOKE
TCL → COMMIT, ROLLBACK, SAVEPOINT
```

This sheet is based on the topics and examples present in your uploaded **MySQL class notes ZIP**, including DDL/DML/DQL, operators, constraints, aggregate functions, mathematical functions, date functions, joins, subqueries, TCL/DCL, views, stored procedures, and triggers.

